Configuring Compiler Options with tsconfig.json

In this lesson, we're going to explore an efficient way to manage compiler options by utilizing a configuration file.

Instead of tediously listing all the options on the command line, we can neatly place them in a configuration file. The key to this process is using a file known as `tsconfig.json`. This file will serve as a centralized location for all our compiler settings.

Let's begin by performing some preparatory work:

1. **Copy and Paste**: I want to start by copying and pasting my previous project into a new location.
2. **Rename the Project**: Next, I'll rename the project, giving it the name `03-tsconfig`.
3. **Addressing Initial Errors**: Don't worry if the IDE initially shows errors. Simply close the previous files, and those errors should disappear.

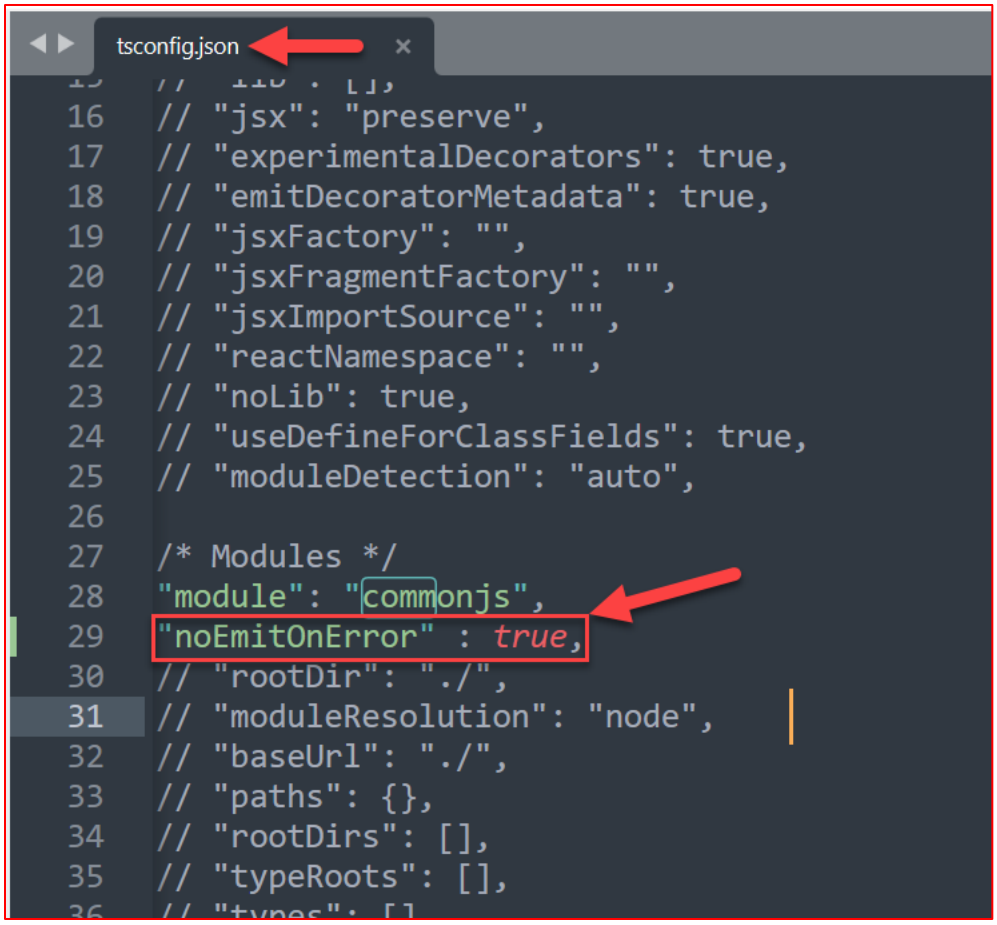
Now, let's transition to our terminal window:

1. **Navigate to the Directory**: Use the command `cd 04-classes/03-tsconfig` to move into the desired directory.
2. **Check the Directory**: As a precaution, make sure you're in the correct directory at this point.
3. **Remove Old Files**: Go ahead and remove the previous JavaScript file `Customer.js`.
4. **Compile the Code**: Upon compilation, you may encounter some compiler errors, indicating that certain accesses are only available when targeting x5 and higher.

These errors tell us that we need to set some specific compiler flags.

Now, we'll make use of the configuration file to address this:

1. **Generate a Template**: By running the command `tsc --init`, we can generate a template configuration file. You should see a message indicating successful file creation.
2. **Open the File**: This file contains the basic configuration along with extensive comments to guide you. It's advisable to refer to the official documentation to understand different configuration items.
3. **Set the Target**: Ensure the target is set to x5, which resolves our previous issue.
4. **Add Additional Configuration**: For example, I'll add the `noEmitOnError` configuration and set its value to `true`.



The advantage of this configuration approach is that we no longer have to manually list all these options on the command line. Instead, we can simply compile, and the process will automatically refer to the configurations defined in this file.

To compile, run `tsc`, and by default, it's going to compile all TypeScript files. To run the application, simply call `node Customer.js`.

In conclusion, using a `tsconfig` file provides an organized and streamlined method for setting up compiler options. It removes the need for manual command-line entries, saving both time and effort. It's an invaluable practice for any developer working with TypeScript and similar languages. Feel free to experiment with these configurations in your projects!